HASTA

Learn the language of hands.

Raspberry	Pale Sky
#C01D5A	#DFF1FD

PROJECT ARCHITECTURE & PRODUCT BLUEPRINT

Locked baseline • Version 1.0 • 30 October 2026

A zero-cost, solo-developer, offline-first Android project with real-time hand tracking, isolated-sign recognition, and reference-based sign-language learning.

ARCHITECTURE BASELINE — NOT A CLAIM OF COMPLETED IMPLEMENTATION

Document control

Item	Value
Product	HASTA — Learn the language of hands.
Document	Project architecture and product blueprint
Version	1.0 — locked architecture baseline
Date	30 October 2026
Platform	Android only; native Kotlin
Delivery model	Solo developer + OpenCode coding agent
Budget assumption	Zero-cost tools and local-first execution; variable free-tier limits must be rechecked
Initial language	Indian Sign Language (ISL)
Release boundary	Isolated signs; not continuous sentence translation
Status	Approved for M0 implementation; choices after M0 remain evidence-driven

How to read this document

This is the shared source of truth for product intent, architecture, research constraints, milestones and acceptance criteria. It combines the locked decisions from the HASTA planning discussion with current official technical references. Where a technology or parameter remains experimental, it is labelled as a candidate or an initial target rather than a measured result.

Important status rule

A feature in this document is a planned capability unless its milestone has been implemented and verified. FPS, model accuracy, model size, battery use and feedback quality are targets to measure, not existing results.

Contents

- 1. Executive summary and product definition
- 2. Vision, users and product boundaries
- 3. Product experience and navigation
- 4. System architecture
- 5. Camera and hand-perception pipeline
- 6. Landmark contract and features-v1
- 7. Recognition model and runtime
- 8. Dataset and data strategy
- 9. Learning content system
- 10. Practice assessment and feedback
- 11. Android application architecture
- 12. HASTA visual identity and design system
- 13. Repository structure and agent workflow
- 14. Development milestones M0–M5
- 15. Evaluation, testing and release gates
- 16. Privacy, licensing and responsible use
- 17. Risks and mitigations
- 18. Definition of done and locked decisions
- 19. Glossary
- 20. References and source notes

1. Executive summary

HASTA is an offline-first Android application for learning and practising sign language. Its first language pack is Indian Sign Language (ISL). The product combines structured lessons and reference demonstrations with a live camera experience that tracks both hands, recognizes a limited vocabulary of isolated signs, and—only for validated signs—provides conservative feedback about a learner's performance.

The architecture is deliberately lean: Kotlin and Jetpack Compose for the Android app; CameraX for camera input; MediaPipe Hand Landmarker for the initial two-hand landmark pipeline; Python and PyTorch for model research; and LiteRT as the proposed on-device inference runtime, subject to model-conversion and device validation. Room stores structured progress locally, DataStore stores small preferences, and lesson/model assets are stored on-device.

Product promise

HASTA helps a learner see a sign, practise it in front of the camera, receive feedback when the evidence is reliable, and build confidence through repeated learning — without requiring an internet connection for core features.

Product at a glance

Dimension	Locked v1 decision
Platform	Android only
Implementation	Native Kotlin; Jetpack Compose; single Gradle module at first
Core language	ISL first; future language-pack format is allowed, but no second language in v1
Tracking	Hands-only baseline; up to two hands
Perception	MediaPipe Hand Landmarker candidate; benchmark on actual phone
Recognition	One temporal classifier; compare GRU and 1D temporal CNN
Input	Hand landmark sequences and derived features; no raw-video model
Inference	On-device LiteRT candidate; export compatibility must be tested
Learning	Dataset/dictionary-driven lessons and curated curriculum structure
Feedback	Per-sign capability gated by tracking confidence and expert validation
Storage	Room + DataStore + local assets
Network	No account, server, cloud inference or sync in v1
Recognition claim	Recognized isolated signs; not full sentence translation
Budget	Zero-cost target; no paid API or hosted inference

What makes the product distinctive

- Learning and recognition share one hand-perception pipeline, but use different decision logic.
- A live camera practice experience is a central product feature, not a separate developer demo.
- Feedback is based on measurable, reference-aligned movement parameters and is disabled when evidence is insufficient.
- The project is designed around reproducible feature contracts and golden vectors so Python research and Kotlin inference remain consistent.
- Offline operation and local-first data handling are core architecture decisions, not later optimizations.

2. Vision, users and product boundaries

2.1 Vision

Make sign-language learning more interactive and accessible by giving learners a structured way to study signs and a private, on-device space to practise them. The product should be useful even when a network connection is unavailable.

2.2 Intended initial users

User group	Need HASTA addresses
Beginners learning ISL	A structured path through vocabulary, demonstrations, short practice sessions and revision.
Family members and friends	A practical way to learn useful signs and practise respectfully.
Students and educators	An offline learning aid with organized content and self-assessment.
Deaf users who want to explore the learning tools	A local reference and practice tool; the product must be designed with Deaf community input and must not assume all Deaf users have the same needs.

2.3 Product principles

- Offline by default: core learning, tracking, recognition and progress work locally.
- Hands-first: hand-only landmark features are the v1 baseline.
- Evidence before complexity: benchmark on the real phone before investing in advanced models or UI breadth.
- Conservative feedback: do not invent corrections when tracking or reference evidence is weak.
- Language integrity: ISL has its own vocabulary, grammar, variation and context; English word mapping is not a complete language model.
- Reproducible engineering: Python and Kotlin use the same versioned feature definitions and golden test vectors.
- Respectful co-design: Deaf and fluent ISL contributors should shape vocabulary, examples and feedback, not only test the finished app.

2.4 V1 scope

Included in v1	Explicitly outside v1
ISL vocabulary and a focused validated sign set	ASL or any second sign language
Static and dynamic isolated signs through one temporal model	Continuous sentence recognition or machine translation
Both-hand tracking where visible and supported by the sign	Face expression, mouth pattern or full-body language interpretation
Text output and optional device speech	Cloud inference, remote APIs or mandatory login
Reference demonstrations, lessons and quizzes	Formal certification or interpreter-grade claims
Parameter-based practice feedback for a validated subset	Unvalidated automatic coaching for every dictionary entry

Terminology guardrail

The app should say “Recognized sign” or “Recognized signs”. Do not call v1 a sign-language translator. Full sign-language meaning can depend on grammar, facial expression, body movement, spatial context and discourse that are outside the hands-only isolated-sign scope.

2.5 Future extensibility

The content format should allow additional language packs later. This does not mean mixing ISL and ASL signs into one classifier in v1. Each future language pack must have its own verified vocabulary, meaning metadata, lessons, reference assets, model labels and validation evidence.

3. Product experience and navigation

3.1 Main navigation

Destination	Purpose	Primary actions
Home	Resume learning and make the next useful action obvious.	Continue lesson; daily practice; quick launch Live Sign Coach.
Learn	Browse structured ISL courses and vocabulary.	Open module; search/browse signs; watch demonstration; take quiz.
Practice	Practise signs with the camera and test recall.	Live Sign Coach; guided practice; random challenge; retry.
Profile	Show local progress and settings.	Mastery, practice history, preferences, theme, language pack, privacy controls.

3.2 Core learner journey

1. Open HASTA without account creation.
2. Choose the ISL beginner path or browse available lessons.
3. Open a lesson with the sign meaning, context and reference demonstration.
4. Watch the sign at normal speed and optionally slow it down or loop it.
5. Enter Practice and allow camera permission.
6. The camera tracks both hands and displays an unobtrusive skeleton overlay.
7. The learner performs the isolated sign.
8. If the tracking and sign-level evidence are sufficient, HASTA shows recognition or an approved correction; otherwise it asks the learner to retry or compare with the reference.
9. The attempt and mastery state are saved locally.
10. A rule-based spaced-revision schedule brings back signs that need practice.

3.3 Live Sign Coach concept

Live Sign Coach is the signature interaction. It should feel like a carefully designed camera product rather than a debug screen. The live view should give the learner enough context to position their hands, but avoid cluttering the signing area with excessive labels, graphs or permanent confidence numbers.

Live state	UI behaviour
Ready	Camera permission and positioning guidance; simple framing hint.
Tracking	Two-hand skeleton overlay, hand visibility indicator and subtle “tracking” state.
Recording sign window	Clear capture indicator; avoid distracting prompts while signing.
Evaluating	Short, calm progress state while sequence alignment and assessment run.
Recognized	Show sign label and optional speech action.
Feedback available	Show only the validated parameters that can be assessed for that sign.
Low confidence	Explain that the attempt could not be assessed reliably and offer retry.
No hand / occlusion	Show a neutral visibility instruction rather than a linguistic correction.

3.4 Visual and interaction requirements

- Prioritize the camera preview and the learner's hands; controls should stay outside the most important hand area.
- Use a large, clear start/stop practice control and obvious recording state.
- Keep corrections short, specific and non-judgmental.
- Never communicate correctness using colour alone; pair status colours with text or icons.
- Support system font scaling, screen reader labels for controls, touch targets and reduced-motion settings.
- Do not place diagnostic FPS, raw coordinates or internal model confidence on the ordinary learner screen. Keep diagnostics in a developer-only/debug build.

4. System architecture

HASTA uses one Android runtime for the user experience and one Python research workspace for dataset processing and model development. The interface, content system, practice assessment and recognition engine are separate logical areas inside a single Gradle module initially. Modules should be split only if real build or ownership pressure appears.

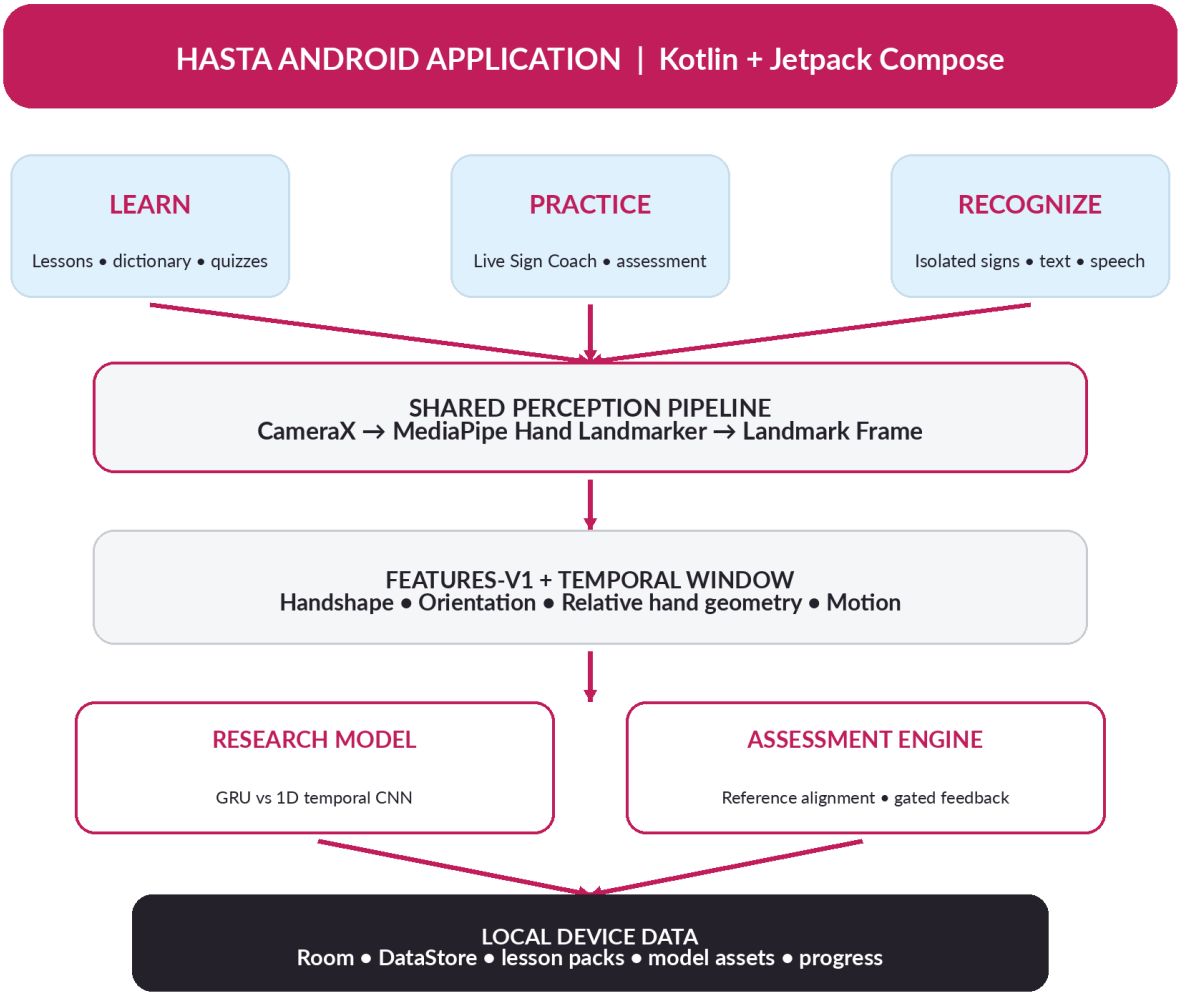


Figure 1. HASTA high-level architecture. The feature and recognition stream is shared; learning and recognition apply different decisions.

4.1 Android runtime responsibilities

Area	Responsibility
UI	Compose screens, navigation, visual states, accessibility and learner interaction.
Domain	Lesson engine, recognition orchestration, assessment rules, revision scheduler and product use cases.
Data	Room DAOs, DataStore preferences, content pack loader, local assets and model manifest.

Area	Responsibility
Perception	CameraX integration, MediaPipe task lifecycle, landmark frame conversion, feature extraction and sequence buffering.

4.2 Python research workspace responsibilities

Area	Responsibility
Dataset audit	Inspect dataset structure, labels, signers, splits, provenance and licence.
Extraction	Read source videos locally and extract MediaPipe landmarks; emit metadata and arrays.
Feature engineering	Implement the exact `features-v1` transformation and export golden vectors.
Training	Train and compare GRU and 1D temporal CNN baselines.
Evaluation	Signer-independent metrics, per-class analysis, false activation and rejection analysis.
Export	Convert the selected model to LiteRT, compare outputs and produce versioned artifacts.

4.3 End-to-end data flow

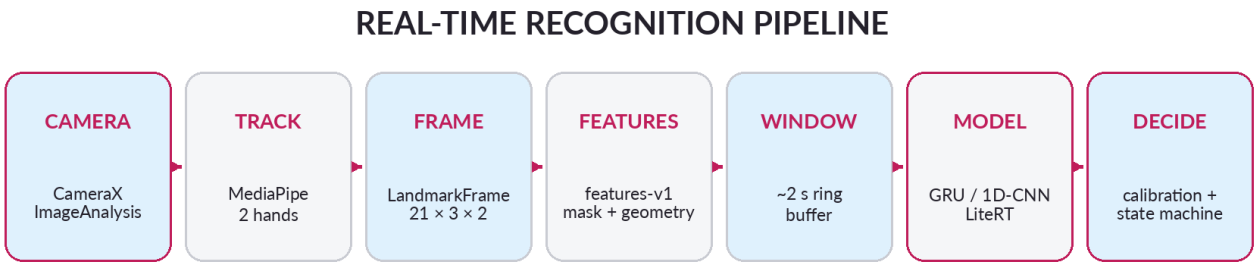


Figure 2. Proposed real-time processing pipeline. The approximately two-second ring buffer is an initial design value to validate during M1/M2.

4.4 Data flow separation

- Live camera frames are processed in memory. The v1 recognizer consumes landmarks/features, not raw RGB frames.
- The recognition engine returns a candidate label, calibrated score and temporal state; the app converts that into user-facing wording.
- The assessment engine compares a completed attempt with sign-specific references and may return parameter observations only when the sign is enabled for feedback.
- Room stores learning progress and summaries. Raw camera frames are not persisted by normal recognition or practice flows.
- Research recordings and volunteer-contributed data use a separate explicit collection/export path; they are not silently collected during everyday app use.

5. Camera and hand-perception pipeline

5.1 CameraX

CameraX is the Android camera abstraction selected for the app. The runtime will use a preview path for the user-facing camera view and an image-analysis path for model input. CameraX supports combining preview and analysis use cases and binds camera behaviour to Android lifecycle state. [1]

- Bind camera resources to the visible lifecycle; release or pause processing when the screen is left.
- Use a backpressure strategy that favours the latest frame for real-time interaction so a slow analyzer does not accumulate stale frames.
- Keep preview smooth even if analysis is throttled. Preview and inference do not need identical frame rates.
- Record frame timestamps and handle rotation and front-camera mirroring consistently.
- Ensure every ImageProxy is closed after processing so the camera pipeline does not stall.

5.2 MediaPipe Hand Landmarker

MediaPipe Hand Landmarker is the initial tracker candidate. Its Android task supports live-stream processing and returns handedness, 21 hand landmarks in image coordinates, and 21 world landmarks per detected hand. Image coordinates contain normalized x/y and relative z; world landmarks are expressed in meters. [2]

Output	Use in HASTA
Handedness	Associate detections with the signer's left/right hand; account for front-camera mirroring.
21 image landmarks	Overlay positions and image-relative spatial features.
21 world landmarks	Compare 3D geometry and orientation where tracker quality supports it.
Detection/presence/tracking confidence	Decide whether a frame is reliable enough for feature generation and assessment.
Timestamp	Preserve temporal order and compute velocity/acceleration.

Hand identity and mirroring

MediaPipe handedness is reported from the subject's perspective. A horizontally mirrored input changes the handedness interpretation. The camera/display transformation must be documented and tested with left- and right-hand signers. [2]

5.3 Two-hand behaviour

The tracker should detect up to two hands. The model and feature contract should represent two canonical hand slots while preserving handedness and explicit presence masks. A sign that needs both hands should not be assessed when one required hand is missing. A one-hand sign may be assessed when its reference definition permits it.

5.4 Hands-only baseline and body-anchor experiment

M0/M1/M2 use hands-only features. Pose landmarks are not included in the baseline model. M2.5 is a controlled experiment that compares the same model and data splits using (a) hands-only inputs and (b) hands plus selected upper-body anchors, if the sign set demonstrates a need for location context.

Keep the hands-only contract stable. If body anchors are adopted later, add a versioned optional feature group such as `body_anchors_v2`; do not silently append zeros or alter the meaning of existing features.

5.5 Failure and degradation behaviour

- No hands: show a framing prompt; do not classify.
- One hand missing: continue only for sign definitions that do not require the missing hand.
- Occlusion or low confidence: pause assessment and ask for a retry.
- Tracking discontinuity: reset or mark the sequence segment; do not join unrelated motion into one sign.
- Thermal or sustained performance pressure: reduce analysis frequency gracefully while preserving a responsive preview; record the degradation in diagnostics.

6. Landmark contract and features-v1

6.1 Why a feature contract matters

The model is trained in Python but runs inside Kotlin. Any mismatch in mirroring, coordinate normalization, slot ordering, missing-landmark handling, feature order or numeric precision can silently damage recognition. `features-v1` is therefore a versioned contract shared by both environments. It is more important than a particular model architecture.

6.2 Per-frame landmark record

Field	Meaning / rule
timestamp_ms	Monotonic frame timestamp used for temporal ordering and derived motion.
left_hand.present	Whether a valid left-hand observation exists for this frame.
right_hand.present	Whether a valid right-hand observation exists for this frame.
handedness	Subject-relative left/right identity from tracker output.
landmarks[21]	Each point stores x, y, z; retain source coordinate-space metadata.
landmark_validity[21]	Per-landmark validity/missingness; do not encode missing values as ordinary zero coordinates.
tracking_confidence	Tracker quality metadata; not equivalent to sign correctness.
image_transform	Rotation and mirror transform used to interpret coordinates.

6.3 Feature groups

Feature group	Examples	Design rule
Local hand geometry	Wrist-relative points, palm-scale normalization, finger joint angles, distances between joints and fingertips.	Reduce camera scale/position variation without losing handshape.
Global hand geometry	Hand centre in image/world space, wrist positions, orientation, distance and relative orientation between hands.	Preserve meaningful signing location and inter-hand relationships.
Temporal motion	Displacement, velocity, acceleration, direction changes and duration over time.	Preserve movement needed to distinguish dynamic signs.
Validity and timing	Hand-presence masks, point validity, timestamps and gap indicators.	Missingness is explicit and must not masquerade as a real pose.

6.4 Critical normalization rules

- Keep both raw or minimally transformed coordinates and derived normalized features where useful for debugging and ablation tests.
- Wrist-relative coordinates may describe local finger shape, but must not replace all global coordinates; doing so could erase sign location and trajectory.
- Use a stable scale reference such as palm geometry; define the exact calculation in the contract and test edge cases.
- Preserve subject-relative handedness. Do not reorder hands based only on screen x-coordinate.
- Represent missing hands and landmarks with explicit masks and documented fallback behaviour.
- Document coordinate units, frame dimensions, mirroring, rotation, clipping and normalization constants.

6.5 Golden vectors

For each fixed input fixture, Python produces the expected feature vector and a Kotlin unit test must reproduce it within a documented floating-point tolerance. The test suite must include both hands, one hand missing, no hand, mirrored frames, rotated frames, extreme scale, partial occlusion, and sequence gaps.

Non-negotiable contract rule

Never edit `features-v1` without updating its specification, regenerating golden vectors, updating the version or migration notes, and passing Python/Kotlin parity tests. CI should run the golden-vector tests at the beginning of the verification sequence.

6.6 Sequence window

The current design uses a ring buffer of approximately two seconds. This is a starting window, not a universal sign-duration rule. M1/M2 should measure natural sign durations in the chosen vocabulary and compare fixed-length sampling, padding/masking and adaptive segmentation. The exact frame count and resampling policy are deliberately not locked before data and device evidence exist.

7. Recognition model and mobile runtime

7.1 One temporal model in v1

The earlier idea of a separate static model and dynamic model router is removed from the v1 architecture. A single compact temporal classifier receives a sequence of hand features. Static signs are represented by low-motion/stable sequences; dynamic signs are represented through changing handshape and trajectory.

7.2 Baseline comparison

Candidate	Why compare it	Expected trade-off to measure
GRU	Compact recurrent baseline for ordered sequence features.	Accuracy across variable duration vs sequential inference cost and export compatibility.
1D temporal CNN	Learns local movement patterns with convolution over time.	Parallel computation and latency vs temporal receptive field and padding/window sensitivity.

The winning model is selected using signer-independent validation, runtime benchmarks and export parity—not by training accuracy alone. Random Forest may be retained as a Python-only baseline if useful; it is not the mobile deployment target in this locked plan.

7.3 Class and state handling

- Vocabulary classes: 20–30 signs for the first recognition milestone.
- Idle class: frames or windows in which no sign is being performed.
- Transition handling: do not emit a sign repeatedly while the learner moves between signs.
- Unknown rejection: use calibrated confidence and validation-derived thresholds; unknown is not simply another label guessed from the trained vocabulary.
- Temporal state machine: candidate → stable prediction → emitted sign → cooldown/transition → ready.
- Deduplicate repeated predictions from one gesture and ensure that a single sign is not emitted multiple times.

7.4 Confidence layers

Signal	Question it answers
Tracking confidence	Did the hand tracker follow the hand reliably?
Landmark validity	Are the required points present and plausible?

Signal	Question it answers
Model probability / calibrated score	Does the sequence resemble a known sign in the trained vocabulary?
Temporal stability	Has the candidate persisted sufficiently to emit one result?
Assessment confidence	Is the reference comparison reliable enough to make a teaching correction?

7.5 LiteRT deployment

LiteRT is the proposed on-device model runtime. Google's current documentation describes LiteRT as an on-device inference engine and recommends the newer CompiledModel API for modern hardware acceleration, while retaining the Interpreter API for backward compatibility. [3] HASTA should use the simplest compatible path that works reliably on the supported Android devices; do not add an accelerator dependency before the CPU baseline is stable.

Google documents a PyTorch-to-LiteRT conversion workflow, but the model must meet the converter's export requirements. Conversion must be demonstrated for the chosen GRU or temporal CNN before runtime is frozen. [4] The conversion step must include output parity checks against PyTorch and tests on the actual phone.

7.6 Model artifact and manifest

```
models/
  isl/
    model.tflite
    manifest.json
    label_map.json
    feature_contract.json
    calibration.json
```

The manifest should record model ID, language, vocabulary version, feature contract version, training dataset identifiers, training code revision, class map hash, input shape, output interpretation, runtime version, quantization mode, model checksum and evaluation summary. It must fail closed when the app loads an incompatible feature contract or label map.

7.7 Optimization policy

- First measure the unoptimized CPU baseline.
- Then test frame-rate throttling, window size and sequence resampling.
- Only then consider quantization or hardware acceleration, checking for accuracy regressions and device-specific behaviour.
- Do not introduce self-supervised pretraining, knowledge distillation, early exit or a larger Transformer until evidence identifies a baseline limitation.

8. Dataset and research data strategy

8.1 Data sources to audit

Source	Known value	HASTA use and restriction
ISLRTC ISL Dictionary	Official FAQ describes 10,000 terms, video signs and English/Hindi equivalents across six broad categories, including everyday, academic, legal/administrative, medical, technical and agricultural terms. Regional variants and synonyms are included. [7]	Primary vocabulary/reference source. ISLRTC says research, teaching and ISL-technology development use is allowed with acknowledgement, but data must not be resold or used for profiteering. Obtain/confirm rights before bundling assets, especially if commercial distribution is contemplated.
ISLRTC Basic ISL self-learning course	Ten modules include greetings, self-introduction, commonly used words, descriptions, basic grammar and situation-based conversation; modules include quizzes. [8]	Use as a curriculum reference and learning-structure blueprint. Do not assume course media can be redistributed; check separate rights.

Source	Known value	HASTA use and restriction
Amrita SLR — Real Time ISL Dataset	Published 20 May 2025; 50 ISL signs, approximately 16 samples per class, 7 signers, 1920×1080 at 28 FPS, average 56 frames / 2.57 seconds; CC BY 4.0. [9]	Candidate training/evaluation source. Verify exact archive contents and annotations. Provide attribution and indicate changes as required by CC BY 4.0.
INCLUDE / CISLR and other ISL sources	Mentioned as candidates in the planning notes; suitability and terms are not yet established.	Audit each source for sign language, labels, signer metadata, sample quality, licence and compatibility. Do not assume it is approved for training or redistribution.

8.2 Dataset audit checklist

- Identify whether samples are isolated signs, continuous signing, images or videos.
- Record exact vocabulary label, gloss, translation and contextual meaning.
- Check whether the same English gloss maps to multiple ISL signs or context-specific variants.
- Record signer IDs, recording sessions, camera device, view, lighting and other available metadata.
- Check whether both hands are visible and whether hand occlusion is common.
- Confirm licence, attribution requirements, derivative-work terms and redistribution restrictions.
- Separate training rights from permission to package original video assets in an APK.
- Record dataset version, DOI or source URL, download date, archive checksum and any transformations.

8.3 Split and leakage rules

- Split by signer whenever signer IDs exist; no signer should appear in both train and final test sets for signer-independent evaluation.
- If several clips come from one continuous recording/session, keep the group together in one split.
- Split before augmentation, window extraction or feature generation to prevent near-duplicate leakage.
- Use validation data for calibration and threshold selection; preserve a final test set for final reporting.
- Report class counts and signer counts for every split. Do not report a macro-F1 without documenting the split protocol.

8.4 Temporary recorder and extraction

Before the app's contribution UI exists, use a small Python webcam recorder to capture volunteer samples, run hand tracking and save landmark sequences and a manifest. This is a research tool, not a user-facing product feature. Keep raw video only when a separate consent and research need explicitly require it; the default output should be landmarks and minimal metadata.

```
research/
  extract/       # video/image -> landmark sequences
  features/      # exact features-v1 implementation
  train/         # GRU and 1D-CNN experiments
  eval/          # splits, metrics, confusion matrices
  export/        # PyTorch -> LiteRT and parity checks
```

8.5 Minimum data manifest

```
{
  "sample_id": "pseudonymous-id",
  "dataset_source": "source-name-and-version",
  "language": "ISL",
  "label_id": "stable-sign-id",
  "signer_id": "pseudonymous-signer-id",
  "session_id": "recording-session-id",
  "capture_device": "optional-device-tag",
  "sequence_path": "relative/path.npy",
  "feature_contract": "features-v1",
  "consent_status": "local-research-consented",
  "licence_and_attribution": "source-specific-record"
}
```

9. Learning content system

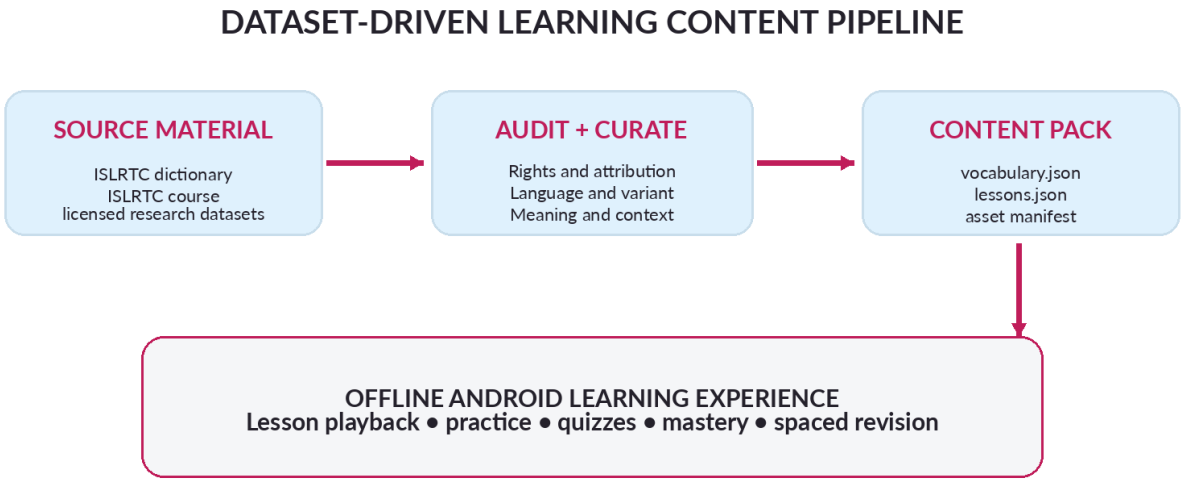


Figure 3. Dataset-driven learning content pipeline. Source assets pass through a rights and metadata audit before becoming offline lesson packs.

9.1 Content-first, not manually recreated

HASTA should reuse existing ISL educational resources where permitted instead of paying to recreate a large sign library from scratch. ISLRTC's dictionary is a particularly valuable starting point: its FAQ describes 10,000 terms, video demonstrations, English/Hindi equivalents, regional signs, synonyms and context-dependent differences. [7]

The ISLRTC basic course provides a ten-module learning structure covering greetings, introductions, vocabulary, descriptions, grammar and situation-based conversations. It also includes quizzes. HASTA can use this as a curriculum reference while building its own offline lesson experience. [8]

9.2 Lesson object

Field	Description
lesson_id	Stable, language-scoped identifier.
language_code	ISL for v1.
title / objective	What the learner will learn.
module_id / order	Curriculum placement and sequence.
sign_ids	One or more vocabulary items taught in the lesson.
reference_asset_id	Video or other permitted demonstration asset.
meaning / usage_note	Context, synonyms, variants and cautions.
practice_mode	Watch, imitate, recognition quiz, recall quiz or camera practice.
assessment_capability	Whether feedback is disabled, comparison-only or expert-validated.
attribution	Dataset/source, creator, licence and required acknowledgement.

9.3 Vocabulary object

Field	Description
sign_id	Stable language-specific identifier; never use English word alone as unique ID.

Field	Description
gloss	Human-readable label used in the app.
meaning	Contextual explanation of the sign.
translations	English/Hindi equivalents where source data provides them.
variants	Regional, alternate or synonym sign variants.
homonym_context	Different meanings that must not be collapsed into one label.
source_reference	Dictionary/dataset identifier and source asset details.
model_label	Classifier class identifier, if included in the trained vocabulary.
feedback_profile	Optional per-sign assessment rules and required tracking conditions.

9.4 Lesson design principles

- Teach meaning and context, not just English-word-to-gesture mapping.
- Distinguish synonyms, regional variants and context-specific signs in the vocabulary model.
- Use a curriculum progression: foundations, common vocabulary, descriptions, grammar awareness and situation-based practice.
- Provide normal-speed and slow/looped reference playback where the asset permits it.
- Keep lesson progress local and usable offline.
- Do not imply a learner has mastered ISL grammar after memorizing isolated signs.

9.5 Asset licensing and fallback

The ISLRTC FAQ permits dictionary use for research, teaching and ISL-related technology development under acknowledgement and no-resale/no-profiteering conditions. [7] This should be treated as a specific constraint, not as general open-source licensing. If HASTA's distribution or monetization model changes, seek written clarification before continuing to distribute or derive assets under these terms.

For any source whose video redistribution permission is unclear, do not package the video into the APK by default. Possible alternatives are linking to the official source when online, creating a new demonstration only with explicit contributor rights, or using a permitted landmark/skeleton demonstration whose source and derivative rights have been checked.

10. Practice assessment and feedback

10.1 Assessment is not classification

Recognition asks: "Which known sign does this sequence resemble?" Assessment asks: "What observable part of this learner's performance differs from a valid reference, and is the evidence strong enough to say so?" HASTA must keep these decisions separate.

10.2 Reference-based comparison

For a sign with approved feedback, capture a complete learner sequence and compare it with multiple valid reference performances. Dynamic Time Warping (DTW) is a candidate for aligning sequences performed at different speeds. It should be evaluated against simpler normalized feature comparisons and not treated as an automatic truth source.

10.3 Assessment dimensions

Dimension	Potential observable measure	Caution
Handshape	Finger joint angles, fingertip distances, palm geometry.	Handshape tolerance must reflect valid signer variation.
Orientation	Palm/hand orientation from supported landmarks.	Occlusion and camera angle can make orientation unreliable.
Relative location	Position of hands and distance between hands.	Hands-only baseline cannot capture all body-relative sign locations.
Movement	Trajectory, path direction, duration and movement extent.	A faster or slower sign should not automatically be considered wrong.
Two-hand relation	Relative timing, distance and orientation between hands.	Do not assess a two-hand sign when one required hand is missing.

10.4 Feedback gating policy

Hard rule

The system emits a correction only when tracking confidence across the relevant frames, landmark validity, reference alignment quality and the sign's own feedback profile all pass their thresholds. Otherwise HASTA says "Could not assess this attempt reliably. Please retry."

Sign feedback state	Permitted behaviour
Not validated	Recognition only; offer "compare with reference". No corrective claim.
Reference exists, feedback not reviewed	Allow playback and side-by-side visual comparison only.
Feedback validated for selected dimensions	Emit only the approved dimension-level feedback when signal quality passes.
Low tracking/reference confidence	Retry; no correction.
Conflicting or ambiguous variant	Show the available variants/context; do not label one performance incorrect without evidence.

10.5 Example feedback language

Evidence	Suitable wording
Tracking weak	"I could not track both hands clearly. Please try again."
Path consistently shorter than accepted reference range	"Your movement was shorter than the reference."
Hand-to-hand spacing outside validated tolerance	"Try placing your hands a little closer together."
No reliable assessment	"Compare your attempt with the reference demonstration."
Recognition candidate rejected	"I could not identify this sign with enough confidence."

10.6 Personalized revision

Use a transparent, rule-based spaced-repetition scheduler (SM-2 is the proposed starting point). The scheduler should use lesson completion and practice outcomes to bring difficult signs back sooner, while increasing the interval for consistently recalled items. Do not add a neural recommender or LLM for v1.

11. Android application architecture

11.1 Technology choices

Layer	Technology	Role / status
Language	Kotlin	Locked.
UI	Jetpack Compose + Material 3	Locked; customize the brand theme and components. [5]
Camera	CameraX	Selected camera API; preview + analysis lifecycle. [1]
Hand tracker	MediaPipe Hand Landmarker	Initial candidate; benchmark and verify Android dependency/device compatibility. [2]
ML training	Python + PyTorch	Research workspace.
Model runtime	LiteRT	Proposed; prove export/inference for selected temporal model. [3][4]
Local database	Room	Structured progress and local content state. [6]
Preferences	DataStore	Small settings and preference objects. [7]
Media playback	Media3	Local lesson video playback.
Speech	Android TextToSpeech	Optional output; offline voice depends on installed voice data.
Async	Coroutines + Flow	Camera/worker orchestration and state streams.
Dependency injection	Manual constructors + AppContainer	Keep early setup simple; avoid extra DI/KSP moving parts.

11.2 Single Gradle module at first

Use one application module with strict package boundaries. Do not begin with a large multi-module Gradle setup. Split modules only when code ownership, compile time, testing or dependency boundaries justify it.

```
android/app/src/main/java/<package>/
  ui/          # Compose screens, navigation, components
  domain/      # use cases, entities, policies
  data/        # Room, DataStore, content/model loaders
  perception/  # CameraX, MediaPipe, feature pipeline
```

11.3 Local persistence

Room is intended for structured on-device data and provides query verification and migration support. [6] Use it for lesson state, progress summaries, practice attempts and revision schedules. DataStore is better suited to small settings and typed preferences; Google notes that large or relational data belongs in Room rather than DataStore. [7]

Store	Data
Room	Modules, lesson completion, sign mastery, practice summaries, revision schedule, feedback availability, local contribution sample metadata.
DataStore	Selected theme, audio preference, selected language pack, onboarding flags, camera preferences.
App assets/files	Model files, model manifests, vocabulary/lesson JSON, downloaded/allowed media and attribution records.

Store	Data
Temporary memory	Live camera frames, landmark frames, feature windows and transient recognizer states.

11.4 Offline state model

- No login gate and no account service in v1.
- No backend, cloud inference, telemetry service or mandatory network check.
- Core screens must open with airplane mode enabled once the app and selected packs are installed.
- A downloaded content pack is versioned; incomplete or incompatible packs must not silently replace an existing working pack.
- TTS is optional and depends on an installed offline voice. Text recognition remains available when speech is unavailable.

11.5 Camera lifecycle and concurrency

- Keep camera analysis off the main UI thread.
- Avoid unbounded frame queues; stale frames are less useful than the latest frame for live practice.
- Ensure deterministic stop/release behaviour when leaving Practice, rotating the device or the app entering the background.
- Do not share mutable frame buffers between tracker, UI and model without clear ownership.
- Keep the UI state immutable and publish only the minimal overlay/result data required for rendering.

12. HASTA visual identity and design system

12.1 Brand

Name: HASTA. Tagline: “Learn the language of hands.” The identity should feel modern, calm, confident and welcoming. It should communicate learning and communication without relying on generic accessibility clip art or a medicalized visual language.

RASPBERRY	PALE SKY
#C01D5A	#DFF1FD

12.2 Colour roles and usage

Role	Value	Use
Primary brand / action	#C01D5A	Primary actions, selected states, key emphasis, brand moments.
Secondary brand / container	#DFF1FD	Calm lesson surfaces, supporting containers and selected highlights.
Main text	#24212B	Body copy and most high-priority text.
Muted text	#696574	Secondary copy where contrast remains sufficient.
Main surface	#FFFFFF	Primary content surface.
Success	#23845A	Success state plus explicit text/icon.
Warning	#C58B24	Warning state plus explicit text/icon.

The approximate 70/20/10 distribution (neutral / Pale Sky / Raspberry) is a visual starting guideline, not a strict constraint. Camera screens may be predominantly dark for visibility and focus.

12.3 Contrast calculations

The following ratios were calculated from the sampled brand colours using the WCAG relative-luminance contrast formula. WCAG 2.2 AA requires 4.5:1 for normal text and 3:1 for large text, subject to the criterion's stated exceptions. [10]

Foreground / background	Contrast	Design decision
White on Raspberry	5.89:1	Passes AA for normal text.
Raspberry on Pale Sky	5.09:1	Passes AA for normal text.
Charcoal #24212B on Pale Sky	13.66:1	Strong text pairing.
Muted #696574 on Pale Sky	4.88:1	Passes AA; avoid very small/thin font weights.
White on Pale Sky	1.16:1	Do not use for text.

12.4 Dark theme

- Create a separate tonal palette; do not simply invert light mode.
- Keep Raspberry as a controlled accent rather than a large, high-luminance dark-mode surface.
- Use Pale Sky for selective foreground accents or calm emphasis only after contrast validation.
- Test each text/background pair and all interactive states independently.
- Do not let wallpaper-based dynamic colour replace the HASTA brand palette; no user toggle is planned for dynamic brand recolouring.

12.5 One-page visual specification

Token	Initial specification to document
Typography	Lato/Roboto-compatible system sans or selected app font; define display, headline, title, body and label styles with explicit line heights and weights.
Spacing	Consistent 4 dp base grid; document the small set of spacing tokens used in the app.
Shapes	Small/medium/large/extra-large corner-radius tokens; avoid over-rounding every element.
Motion	Define short, purposeful transitions; support reduced-motion settings.
Elevation	Use restrained surface hierarchy; avoid excessive shadows and gradient effects.
Components	Six core components: buttons, surfaces/cards, lesson card, progress indicator, navigation/top bar, camera overlay/feedback panel.

12.6 Design quality bar

- One distinctive Live Sign Coach screen should be polished before creating many secondary screens.
- Use whitespace and strong typography hierarchy rather than decorative gradients and repeated cards.
- Use native Android navigation, system insets, permission patterns and accessibility conventions.
- All camera states, permission denial, no-data, loading, retry, error and success states must have designed UI.
- The interface must still work under large font scaling and with screen readers for non-camera controls.

13. Repository structure and OpenCode workflow

13.1 Lean repository layout

```
hasta/
  AGENTS.md
  README.md
  docs/
    spec/
      features-v1.md
```

```
    visual-spec.md
    product-scope.md
  architecture/
    system-overview.md
    data-flow.md
  decisions/
    ADR-0001-hands-only-m0.md
    ADR-0002-temporal-model.md
    ADR-0003-offline-first.md
  tasks/
android/
  app/src/main/java/<package>/
    ui/
    domain/
    data/
    perception/
research/
  extract/
  features/
  train/
  eval/
  export/
contracts/
  golden_vectors/
  pack.schema.json
  model_manifest.schema.json
content/
  isl/
    vocab.json
    lessons.json
    attribution.md
models/
  isl/
    manifest.json
scripts/
tests/
```

13.2 AGENTS.md rules

- This architecture and docs/spec are the source of truth; do not redesign the product while implementing a task.
- Start with one Gradle module; do not add modules unless the task explicitly authorizes an ADR.
- Do not change feature ordering, normalization, coordinate transforms or masks without updating features-v1 and golden vectors.
- Golden-vector parity tests run first in CI and must pass before model/integration test suites.
- No extra dependencies or network services without an explicit decision.
- Keep tasks small: state goal, files allowed to change, acceptance tests and excluded scope.
- Plan first, wait for review, then implement; use small diffs and commit each green task.
- Do not invent benchmark or model metrics.
- Do not store or upload user video/landmark data without explicit consent and an approved data path.
- UI work must use HASTA design tokens and states; do not introduce a new colour palette ad hoc.

13.3 Task workflow for OpenCode

11. Write a task file with a narrow goal, specific files, invariants, acceptance criteria and test commands.
12. Ask OpenCode for a plan and inspect its assumptions before implementation.
13. Have the agent add or update tests first for behaviour that can be tested.
14. Implement the smallest change set and inspect the diff.
15. Run golden vectors, relevant unit tests, Android build and device test as required.
16. Commit only when the relevant checks are green; start a fresh session for the next task.

14. Development milestones M0–M5

The milestones are dependency-ordered. Engineering and product-design tracks may run in parallel only where they do not obscure the main feasibility gate. Milestone completion means its acceptance evidence exists; it does not mean every aspirational metric has been achieved.

M0 — Device feasibility spike

Goal: prove that CameraX and MediaPipe hand tracking can run continuously on the actual Android phone and show a skeleton overlay.

Work item	Acceptance evidence
Minimal Compose shell with camera permission and preview.	Permission granted/denied flows tested; preview works on physical device.
CameraX preview + ImageAnalysis.	Frames reach analyzer; analyzer closes frames and does not stall preview.
MediaPipe Hand Landmarker with two-hand configuration.	Handedness and 21-point landmarks shown for supported hands.
OverlayView / Compose overlay.	Landmarks align correctly under front-camera mirroring and device rotation.
Ten-minute continuous benchmark.	Report includes device/OS/build, effective FPS and frame intervals, latency percentiles, peak memory, battery change and thermal state.
Sustained target.	15–20 FPS is an initial viability target to measure, not a claimed result or unconditional release guarantee.

M0.5 — Brand and camera shell

Work item	Acceptance evidence
HASTA theme tokens.	Exact Raspberry #C01D5A and Pale Sky #DFF1FD; semantic supporting roles and dark tonal palette documented.
One-page visual specification.	Typography, spacing, radius, motion and six core components specified.
Live Sign Coach camera shell.	Designed ready/tracking/no-hand/permission/error states; no debug clutter.
Accessibility pass.	Contrast, touch target, text scaling, screen-reader labels and reduced-motion behaviour reviewed.

M1 — Feature contract and data tooling

Work item	Acceptance evidence
features-v1 specification.	Exact coordinates, slots, masks, units, normalization, order and timing defined.
Python recorder.	A temporary local webcam tool records consented volunteer samples as landmark sequences and manifests.
Extraction scripts.	Dataset/source video can be converted reproducibly to landmark sequences.
Golden vectors.	Fixtures cover edge cases and expected output is versioned.
Kotlin parity.	Kotlin unit tests reproduce Python vectors within a defined tolerance.
CI.	Golden-vector tests run at the beginning and fail the pipeline on mismatch.

M2 — First recognizer

Work item	Acceptance evidence
Initial vocabulary.	20–30 carefully selected ISL signs with stable IDs and documented sign meanings.
GRU and 1D temporal CNN.	Same data split and evaluation protocol used for comparison.

Work item	Acceptance evidence
Idle and transition handling.	State machine prevents repeated emissions and false activations during transitions.
Unknown rejection.	Threshold calibrated on validation data; false acceptance and false rejection reported.
LiteRT export.	Converted model outputs match the PyTorch reference within documented tolerance.
Live run.	Model operates with camera landmarks on the physical phone and logs end-to-end latency.

M2.5 — Body-anchor controlled experiment

Train/evaluate the same model using hands-only features and hands-plus-selected-body-anchor features. Use the same signer-independent splits, training protocol and metrics. Compare per-sign gains and costs. Record the decision in an ADR. If the gain does not justify complexity, retain hands-only.

M3 — Learning product shell

Work item	Acceptance evidence
Content pack schema and ISL starter pack.	Source, licence, attribution, variants, context and asset rights are recorded.
Learn experience.	A learner can complete an offline lesson using local permitted content.
Room + DataStore.	Progress and settings survive process restarts.
SM-2 revision.	Review intervals update deterministically from quiz/practice outcomes.
Contribution mode.	Explicit opt-in, local by default, metadata and delete-before-export work correctly.

M4 — Sign-specific assessment

Work item	Acceptance evidence
Reference profiles.	Multiple valid reference performances for each enabled sign.
Parameter measures.	Handshape, orientation, location, movement and two-hand relation used only where observable.
Confidence gate.	Low tracking/reference quality returns retry, not correction.
Per-sign validation.	Feedback enabled only after expert/fluent-signer review of the sign profile.
Agreement report.	Human reviewers evaluate whether system feedback is correct, useful and appropriately cautious.

M5 — Beta

Work item	Acceptance evidence
Vocabulary expansion target.	50–150 signs is a planning target; each sign must have sufficient signer diversity and validation.
Real tester cohort.	10+ testers as an initial target, including Deaf users and fluent ISL users.
Accessibility and usability.	Camera guidance, content readability, feedback language and navigation reviewed.
Release build.	Signed APK distributed through GitHub Releases for beta; install/update instructions documented.
Evidence pack.	Performance report, signer-independent recognition results, assessment review and known limitations included.

15. Evaluation, testing and release gates

15.1 Recognition evaluation

Metric	Why report it
Signer-independent macro-F1	Balances classes without letting large classes dominate and tests generalization to unseen signers.
Per-class precision and recall	Shows which signs are reliably detected and which are confused.
Confusion matrix	Identifies visually or temporally similar signs.
Unknown false-accept rate	Measures how often an out-of-vocabulary sequence is incorrectly mapped to a known sign.
Unknown false-reject rate	Measures how often a known sign is rejected.
False activations per minute	Measures nuisance outputs during idle or natural movement.
Per-signer breakdown	Reveals performance variation and potential demographic/device biases.

15.2 On-device performance evaluation

Metric	Measurement approach
Effective FPS	Completed analysis frames per second; report average and distribution of frame intervals.
End-to-end latency	Timestamp from camera frame acquisition to stabilized recognition result; report p50/p95.
Peak memory	Record app/process memory during continuous camera + recognition use.
Battery impact	Record battery level before/after a controlled session and charging state.
Thermal state	Record thermal status before, during and after sustained test where Android API/device supports it. [11]
Frame drops / backpressure	Record submitted, processed, dropped and stale frames.
Crash/stall behaviour	No app crash, deadlock, camera stall or unbounded queue during the ten-minute run.

15.3 M0 benchmark protocol

17. Record phone model, Android version, app build/commit, tracker/model versions and camera used.
18. Run a ten-minute continuous session on the physical phone, not only an emulator.
19. Use a repeatable test environment: consistent camera distance, lighting, screen brightness, foreground state and starting battery range.
20. Record effective FPS, frame interval p50/p95, tracking latency and end-to-end latency p50/p95.
21. Record memory samples, battery start/end, charging state and thermal status samples.
22. Repeat at least once after a cold start; document variance between runs.
23. Keep raw logs and summarize results in `docs/research/m0-device-benchmark.md`.

15.4 Teaching-quality evaluation

Measure	Question
Feedback correctness	Would fluent ISL reviewers agree that the stated correction is accurate?
Feedback coverage	For what fraction of attempts can the system safely provide feedback?
Retry appropriateness	Does the app ask for a retry when tracking/reference quality is insufficient?
Learner improvement	Do repeated practices improve performance against the validated reference rubric?

Measure	Question
Usability and tone	Do learners understand the feedback, and does it avoid discouraging or stigmatizing language?

15.5 Test pyramid

- Unit tests: feature transforms, normalization, DTW/alignment, state machine, threshold logic, scheduler and content schemas.
- Golden-vector tests: Python/Kotlin parity; first CI gate.
- Model tests: expected labels, shape, output parity and quantization regression.
- Instrumented tests: permissions, camera lifecycle, orientation, overlay, local storage and offline operation.
- Physical-device tests: ten-minute camera session, heat, battery, frame drops and low-light/occlusion scenarios.
- Human review: vocabulary accuracy, feedback validity, Deaf user experience and respectful language.

16. Privacy, licensing and responsible use

16.1 Privacy by design

- Normal recognition processes frames in memory; do not save video by default.
- Practice attempts store only the minimum progress summary needed by the app, unless the user explicitly starts a research contribution flow.
- Contribution mode is opt-in, clearly explains what is stored and why, and defaults to local-only storage.
- Separate consent is required before any contribution leaves the device or is shared with a research collaborator.
- Use pseudonymous signer IDs; avoid names, phone numbers, exact locations or unnecessary personal attributes in research manifests.
- Provide deletion of individual contribution samples before export and document how exported copies are handled.
- Do not add analytics or cloud telemetry in v1. Any future change requires a separate decision and privacy review.

16.2 Content and dataset licensing

Dataset licences apply to specific assets and uses. HASTA must preserve source identifiers, licence versions, attribution and transformation history. A dataset being publicly downloadable does not mean its videos can automatically be packaged or resold.

Resource	Required handling
ISLRTC dictionary	Acknowledge ISLRTC and the Department of Empowerment of Persons with Disabilities, Ministry of Social Justice and Empowerment, Government of India. FAQ states the data are not to be resold or used for profiteering. Obtain written clarification for any future commercial distribution/monetization and verify rights before bundling video assets. [7]
ISLRTC course	Use as a curriculum reference only unless course-media reuse/redistribution terms are confirmed separately. [8]
Amrita SLR CC BY 4.0	Provide attribution, identify the dataset, link the licence and indicate modifications. Keep the dataset DOI and version in manifests. [9]
Volunteer contributions	Use a contributor consent/permission record defining research, model-training, app use, attribution preference, withdrawal and deletion conditions.

16.3 Responsible product claims

- Do not claim full ISL translation or interpreter-level accuracy.
- Clearly state that v1 handles a limited, trained set of isolated signs.
- Disclose that non-manual signals and grammar are outside hands-only recognition scope.
- Show “not recognized” or “retry” rather than making an unsupported prediction.
- Do not present recognition output as authoritative communication in medical, legal, emergency or other high-stakes settings.
- Include Deaf and fluent ISL collaborators early enough to influence product direction and evaluation.

17. Risks and mitigations

Risk	Impact	Mitigation / trigger
Insufficient signer diversity	Model memorizes a few people and fails on new signers.	Treat data diversity as the primary risk. Recruit community collaborators; group splits by signer; show per-signer results; do not inflate sample counts with adjacent frames.
Dataset incompatibility	Labels, clips or annotations do not match isolated-sign recognition.	Audit every dataset before training; keep incompatible material for reference/context only.
ISL dictionary asset rights	Could prevent bundling or commercial use of videos.	Separate source access from redistribution rights; maintain skeleton/own-recording fallback; seek clarification before commercial use.
Hand-only limitations	Some signs rely on body location, face, head or context.	State the limitation; test body anchors at M2.5; do not claim coverage beyond the evaluated vocabulary.
Camera/tracker performance	Low light, occlusion, motion blur or thermal throttling cause tracking loss.	M0 ten-minute benchmark; show retry states; tune frame rate; test across phones and environments.
Python/Kotlin drift	Feature mismatch silently degrades the model.	Versioned feature contract, golden vectors and CI-first parity tests.
Model conversion failure	Selected architecture may not export or may change outputs.	Test export feasibility early in M2; keep CPU reference; choose model based on device and conversion evidence.
False teaching correction	Learner receives incorrect instruction.	Sign-specific validation, multiple references, confidence gates and conservative wording.
Scope creep	Too many models, screens or languages delay shipping.	No second language, accounts, sync, sentence translation or advanced training before v1 evidence.
Free tool quota or API changes	Could disrupt workflow or introduce costs.	Keep tools local/open-source; re-check free-tier limits; no paid service is a runtime dependency.

17.1 Stop/go gates

Gate	Go when	Stop or revise when
M0 → M1	Tracking works for both hands on real hardware and benchmark evidence exists.	Camera stalls, severe thermal degradation, unstable coordinates or device limitations require pipeline adjustment.
M1 → M2	Feature contract and Kotlin parity tests are stable.	Any unresolved feature mismatch or leakage in data pipeline.
M2 → M2.5/M3	A baseline recognizer works live and signer-independent metrics are reported.	High false activation, unusable unknown rejection, or unsupported claims.
M3 → M4	Offline lessons and progress work; feedback profiles have reference material.	Rights unclear or content semantics not validated.
M4 → M5	Feedback review shows acceptable correctness and cautious failure behaviour.	Inaccurate corrections or low agreement; disable feedback for those signs.

18. Definition of done and locked decisions

18.1 Locked architecture decisions

Decision area	Locked baseline
Brand	HASTA — “Learn the language of hands.”

Decision area	Locked baseline
Brand colours	Raspberry #C01D5A; Pale Sky #DFF1FD.
Platform	Android only, native Kotlin.
UI	Jetpack Compose with customized Material 3 foundation.
Project mode	Zero-cost, solo-developer build; OpenCode is a coding agent, not an autonomous product architect.
App architecture	One Gradle module initially; package-by-feature boundaries.
Data/inference	Offline-first; no account, sync, cloud inference or backend in v1.
Language	ISL first; future language pack compatibility only.
Recognition scope	Isolated signs, not sentence translation.
Perception	Hands-only baseline; up to two hands.
Body anchors	Controlled comparison at M2.5; not part of baseline features-v1.
Model	One temporal classifier; compare GRU and 1D CNN.
Runtime	LiteRT proposed; validate converter and runtime for selected model.
Research data	Landmark sequences by default; temporary Python recorder first; app contribution mode later.
Feedback	Sign-specific, confidence-gated and enabled only after validation.
Local data	Room for structured progress; DataStore for small preferences.
Evaluation	Signer-independent split, macro-F1, per-class analysis, unknown rejection, device performance and feedback review.

18.2 Definition of done for a task

- Task has an explicit goal and excluded scope.
- Acceptance criteria are testable and have evidence.
- Feature-contract changes are versioned and golden vectors regenerated.
- Relevant tests pass; failures are reported honestly.
- No unrelated architecture or design changes are included.
- Documentation and source/licence metadata are updated.
- Physical-device behaviour is checked for camera, model and thermal/performance tasks.

18.3 Definition of done for v1 beta

- A user can install and open HASTA without account creation or network access.
- A user can complete a local ISL lesson using permitted assets.
- Live Sign Coach tracks hands on a physical device and presents meaningful tracking states.
- The model recognizes only the documented vocabulary and rejects uncertain/out-of-scope attempts at a measured rate.
- The app labels results as isolated recognized signs, not sentence translation.
- Practice feedback is limited to sign profiles that have passed review and confidence checks.
- Progress and settings persist locally; no raw video is collected by default.
- Performance, recognition and teaching-quality reports document limitations.

19. Glossary

Term	Meaning in HASTA
ISL	Indian Sign Language.
Gloss	A short textual label used to refer to a sign; it is not necessarily a full linguistic translation.

Term	Meaning in HASTA
Landmark	A key point on the hand returned by the tracker.
Handedness	Whether a detected hand is the signer's left or right hand.
features-v1	The versioned, exact transformation from landmark frames to model input features.
Golden vector	A fixed test input and expected feature output used to ensure Python and Kotlin agree.
Signer-independent split	A dataset split where people in the test set do not appear in the training set.
Macro-F1	The unweighted mean of per-class F1 scores.
Temporal window	A sequence of consecutive frames/features used by the model.
GRU	Gated Recurrent Unit, a recurrent neural-network layer for sequence data.
1D temporal CNN	A convolutional model applied across the time dimension of a sequence.
DTW	Dynamic Time Warping; a sequence-alignment method that can compare performances with different durations.
LiteRT	Google's on-device inference framework; model conversion and device support must be verified for HASTA's selected network.
Unknown rejection	A policy that declines to map an uncertain or out-of-vocabulary sequence to a known sign.
SM-2	A classic spaced-repetition scheduling algorithm used as the initial deterministic revision policy.
Assessment profile	A per-sign set of reference data, observable dimensions, confidence requirements and validated feedback text.

20. References and source notes

External sources were checked on 30 October 2026. These references support technology descriptions, content-source facts and licensing cautions. Implementation versions and terms should be rechecked when the corresponding milestone is started.

[1] Android Developers — CameraX architecture
https://developer.android.com/media/camera/camerax/architecture
CameraX use cases, preview and image analysis, controller/provider choices and lifecycle binding.

[2] Google AI Edge — Hand landmarks detection guide for Android
https://developers.google.com/edge/mediapipe/solutions/vision/hand_landmarker/android
Android task, live stream handling, handedness, image/world landmarks and 21 points per hand.

[3] Google AI Edge — Getting started with LiteRT
https://developers.google.com/edge/litert/overview
Current LiteRT positioning, on-device inference and CompiledModel/Interpreter APIs.

[4] Google AI Edge — PyTorch to LiteRT quickstart
https://developers.google.com/edge/litert/conversion/pytorch/overview
PyTorch export compatibility, conversion, serialization and parity validation.

[5] Android Developers — Material Design 3 in Compose
https://developer.android.com/develop/ui/compose/designsystems/material3
Compose theme, color roles, typography scale, shapes and design system foundation.

[6] Android Developers — Save data in a local database using Room
https://developer.android.com/training/data-storage/room
Room for structured local persistence and offline access.

[7] ISLRTC — Frequently Asked Questions
https://islrtc.nic.in/faq/
10,000-term dictionary, categories, synonyms/variants, source access and research/teaching/technology-use conditions.

[8] ISLRTC — Online Basic ISL Course in Self Learning Mode
https://islrtc.nic.in/online-basic-isl-course-in-self-learning-mode/

Ten-module course structure, vocabulary, grammar, situation-based conversation and quizzes.

[9] Mendeley Data — Amrita SLR: Real Time ISL Dataset, Version 1
https://data.mendeley.com/datasets/dtg9s785v8/1
Dataset description, sign/signer/video statistics and CC BY 4.0 licence.

[10] W3C WAI — WCAG 2.2 Contrast (Minimum)
https://www.w3.org/WAI/WCAG22/Understanding/contrast-minimum.html
4.5:1 normal text and 3:1 large text minimum contrast thresholds under SC 1.4.3.

[11] Android Developers — PowerManager API
https://developer.android.com/reference/android/os/PowerManager
Thermal status listener and current thermal status APIs for sustained-device testing.

[12] Android Developers — DataStore
https://developer.android.com/topic/libraries/architecture/datastore
DataStore for small key-value or typed preferences; Room for larger/relational datasets.

Source interpretation notes

- ISLRTC's published terms are reproduced as a project constraint summary, not as legal advice. Confirm the applicable rights in writing if any commercial or monetized distribution is planned.
- The Amrita dataset description is treated as a candidate data source. The actual archive must still be inspected; the dataset description does not prove that every file or label will fit HASTA's task.
- LiteRT and Android documentation can change. The dependency versions and model API must be pinned only when M0/M2 implementation begins.
- All performance values in this blueprint are targets or items to measure unless explicitly attributed to a source. No HASTA model performance has yet been demonstrated.